

BmpImage Project

Generated by Doxygen 1.9.1

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 BMPHeader Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Data Documentation	5
3.1.2.1 fileSize	5
3.1.2.2 fileType	6
3.1.2.3 offsetData	6
3.1.2.4 reserved1	6
3.1.2.5 reserved2	6
3.2 BmpImage Class Reference	6
3.2.1 Detailed Description	7
3.2.2 Member Function Documentation	7
3.2.2.1 createGaussianKernel()	7
3.2.2.2 gaussFilter()	8
3.2.2.3 getHeight()	10
3.2.2.4 getPixel()	11
3.2.2.5 getWidth()	11
3.2.2.6 load()	11
3.2.2.7 rotate90Clockwise()	12
3.2.2.8 rotate90CounterClockwise()	13
3.2.2.9 save()	13
3.2.3 Member Data Documentation	14
3.2.3.1 data	14
3.2.3.2 dibHeader	14
3.2.3.3 header	15
3.2.3.4 height	15
3.2.3.5 width	15
3.3 DIBHeader Struct Reference	15
3.3.1 Detailed Description	16
3.3.2 Member Data Documentation	16
3.3.2.1 bitCount	16
3.3.2.2 colorsImportant	16
3.3.2.3 colorsUsed	16
3.3.2.4 compression	16
3.3.2.5 height	17
3.3.2.6 imageSize	17
3.3.2.7 planes	17

3.3.2.8 size	17
3.3.2.9 width	17
3.3.2.10 xPixelsPerMeter	18
3.3.2.11 yPixelsPerMeter	18
3.4 Pixel Struct Reference	18
3.4.1 Detailed Description	18
3.4.2 Member Data Documentation	18
3.4.2.1 blue	19
3.4.2.2 green	19
3.4.2.3 red	19
4 File Documentation	21
4.1 include/bmp_image.h File Reference	21
4.2 include/struct.h File Reference	21
4.3 include/threads.h File Reference	21
4.3.1 Function Documentation	22
4.3.1.1 thread_1()	22
4.3.1.2 thread_2()	22
4.4 src/bmp_image.cpp File Reference	23
4.5 src/threads.cpp File Reference	23
4.5.1 Function Documentation	23
4.5.1.1 thread_1()	24
4.5.1.2 thread_2()	25
Index	27

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BMPHeader	BMP file header	5
BmpImage	Class for loading, manipulating, and saving BMP images	6
DIBHeader	DIB header	15
Pixel	A single pixel in BGR format	18

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

include/bmp_image.h	21
include/struct.h	21
include/threads.h	21
src/bmp_image.cpp	23
src/threads.cpp	23

Chapter 3

Class Documentation

3.1 BMPHeader Struct Reference

BMP file header.

```
#include <struct.h>
```

Public Attributes

- uint16_t [fileType](#)
- uint32_t [fileSize](#)
- uint16_t [reserved1](#)
- uint16_t [reserved2](#)
- uint32_t [offsetData](#)

3.1.1 Detailed Description

BMP file header.

Contains the file signature, file size, reserved fields, and the offset to the start of pixel data.

Definition at line 16 of file struct.h.

3.1.2 Member Data Documentation

3.1.2.1 fileSize

```
uint32_t BMPHeader::fileSize
```

Total size of the BMP file in bytes.

Definition at line 19 of file struct.h.

3.1.2.2 fileType

```
uint16_t BMPHeader::fileType
```

File type signature. Must be 0x4D42 ("BM").

Definition at line 18 of file struct.h.

3.1.2.3 offsetData

```
uint32_t BMPHeader::offsetData
```

Offset in bytes to the start of the pixel data.

Definition at line 22 of file struct.h.

3.1.2.4 reserved1

```
uint16_t BMPHeader::reserved1
```

Reserved1

Definition at line 20 of file struct.h.

3.1.2.5 reserved2

```
uint16_t BMPHeader::reserved2
```

Reserved2

Definition at line 21 of file struct.h.

The documentation for this struct was generated from the following file:

- [include/struct.h](#)

3.2 BmpImage Class Reference

Class for loading, manipulating, and saving BMP images.

```
#include <bmp_image.h>
```

Collaboration diagram for BmpImage:

Public Member Functions

- void [load](#) (const std::string &filename)
Loads a BMP image from file.
- void [rotate90Clockwise](#) ()
Rotates the image 90 degrees clockwise.
- void [rotate90CounterClockwise](#) ()
Rotates the image 90 degrees counterclockwise.
- void [save](#) (const std::string &filename)
Saves the image to a BMP file.
- void [gaussFilter](#) (int radius, double sigma)
Applies a Gaussian blur filter to the image.
- std::vector< std::vector< double > > [createGaussianKernel](#) (int radius, double sigma)
Creates a 2D Gaussian kernel matrix.
- int32_t [getWidth](#) ()
Returns the width of the image.
- int32_t [getHeight](#) ()
Returns the height of the image.
- std::vector< std::vector< [Pixel](#) > > [getPixel](#) ()
Returns the full 2D pixel matrix.

Private Attributes

- [BMPHeader](#) header
- [DIBHeader](#) dibHeader
- int32_t width
- int32_t height
- std::vector< std::vector< [Pixel](#) > > data

3.2.1 Detailed Description

Class for loading, manipulating, and saving BMP images.

Definition at line 16 of file bmp_image.h.

3.2.2 Member Function Documentation

3.2.2.1 createGaussianKernel()

```
std::vector< std::vector< double > > BmpImage::createGaussianKernel (  
    int radius,  
    double sigma )
```

Creates a 2D Gaussian kernel matrix.

Generates a normalized 2D Gaussian kernel.

Parameters

<i>radius</i>	Radius of the kernel.
<i>sigma</i>	Standard deviation.

Returns

2D vector containing the Gaussian kernel.

The kernel is used for Gaussian blur filtering. The values are computed using the Gaussian function and then normalized so the sum of all weights is 1.

Parameters

<i>radius</i>	Radius of the kernel (defines the size: $2 \cdot \text{radius} + 1$).
<i>sigma</i>	Standard deviation of the Gaussian distribution.

Returns

A 2D vector representing the normalized kernel.

Definition at line 153 of file `bmp_image.cpp`.

```

154 {
155     int size = 2 * radius + 1;
156     std::vector<std::vector<double>> kernel(size, std::vector<double>(size));
157     double sum = 0.0;
158
159     for (int i = -radius; i <= radius; ++i)
160     {
161         for (int j = -radius; j <= radius; ++j)
162         {
163             kernel[i + radius][j + radius] = std::exp(-(i * i + j * j) / (2 * sigma * sigma));
164             sum += kernel[i + radius][j + radius];
165         }
166     }
167
168     for (int i = 0; i < size; ++i)
169     {
170         for (int j = 0; j < size; ++j)
171         {
172             kernel[i][j] /= sum;
173         }
174     }
175
176     return kernel;
177 }
```

Here is the caller graph for this function:

3.2.2.2 gaussFilter()

```

void BmpImage::gaussFilter (
    int radius,
    double sigma )
```

Applies a Gaussian blur filter to the image.

Applies a Gaussian blur to the image using a given radius and sigma.

Parameters

<i>radius</i>	Radius of the Gaussian kernel.
<i>sigma</i>	Standard deviation of the Gaussian distribution.

Each pixel is updated by performing a convolution with the Gaussian kernel. Edges are handled by skipping out-of-bounds coordinates.

Parameters

<i>radius</i>	Radius of the Gaussian kernel.
<i>sigma</i>	Standard deviation (spread) of the Gaussian kernel.

Applies a Gaussian blur filter to the image using the given radius and sigma.

This method applies Gaussian convolution across the image using a multithreaded approach. The image is divided into horizontal blocks (rows), and each thread processes a separate block. The Gaussian kernel is generated once and shared across all threads for efficiency.

Parameters

<i>radius</i>	Radius of the Gaussian kernel.
<i>sigma</i>	Standard deviation for the Gaussian distribution.

Lambda function to process a vertical slice of rows.

Parameters

<i>start</i> _↔ <i>_y</i>	Starting row index (inclusive).
<i>end</i> _↔ <i>_y</i>	Ending row index (exclusive).

Definition at line 227 of file bmp_image.cpp.

```

228 {
229     // Generate the Gaussian kernel once
230     std::vector<std::vector<double>> kernel = createGaussianKernel(radius, sigma);
231
232     // Get the number of available hardware threads
233     int num_threads = std::thread::hardware_concurrency();
234
235     std::cout << "Found " << num_threads << " cores" << '\n';
236
237     if (num_threads == 0)
238     {
239         std::cout << "Didn't find a core, 4 cores will be used\n";
240         num_threads = 4;
241     }
242
243     std::vector<std::thread> threads;
244
245     /**
246      * @brief Lambda function to process a vertical slice of rows.
247      *
248      * @param start_y Starting row index (inclusive).
249      * @param end_y   Ending row index (exclusive).
250      */
251     auto worker = [this, &radius, &kernel](int start_y, int end_y)
252     {
253         for (int y = start_y; y < end_y; ++y)
254         {
255             for (int x = 0; x < width; ++x)

```

```

256         {
257             double sumR = 0.0, sumG = 0.0, sumB = 0.0;
258
259             // Apply convolution using the kernel
260             for (int ky = -radius; ky <= radius; ++ky)
261             {
262                 for (int kx = -radius; kx <= radius; ++kx)
263                 {
264                     int ny = y + ky;
265                     int nx = x + kx;
266
267                     // Only include pixels within image bounds
268                     if (ny >= 0 && ny < height && nx >= 0 && nx < width)
269                     {
270                         sumR += data[ny][nx].red * kernel[ky + radius][kx + radius];
271                         sumG += data[ny][nx].green * kernel[ky + radius][kx + radius];
272                         sumB += data[ny][nx].blue * kernel[ky + radius][kx + radius];
273                     }
274                 }
275             }
276
277             // Clamp results to [0, 255] and write back to the image
278             data[y][x].red = static_cast<uint8_t>(std::min(255.0, std::max(0.0, sumR)));
279             data[y][x].green = static_cast<uint8_t>(std::min(255.0, std::max(0.0, sumG)));
280             data[y][x].blue = static_cast<uint8_t>(std::min(255.0, std::max(0.0, sumB)));
281         }
282     }
283 };
284
285 // Compute the number of rows each thread should process
286 int block = height / num_threads;
287
288 // Launch threads with different row ranges
289 for (int i = 0; i < num_threads; ++i)
290 {
291     int start = i * block;
292     int end;
293
294     // Let the last thread handle any remaining rows
295     if (i == num_threads - 1)
296     {
297         end = height;
298     }
299     else
300     {
301         end = (i + 1) * block;
302     }
303
304     threads.push_back(std::thread(worker, start, end));
305 }
306
307 // Wait for all threads to complete
308 for (auto& t : threads)
309 {
310     t.join();
311 }
312 }

```

Here is the call graph for this function: Here is the caller graph for this function:

3.2.2.3 getHeight()

```
int32_t BmpImage::getHeight ( )
```

Returns the height of the image.

Returns

Image height in pixels.

Definition at line 319 of file bmp_image.cpp.

```

320 {
321     return height;
322 }

```

3.2.2.4 getPixel()

```
std::vector< std::vector< Pixel > > BmpImage::getPixel ( )
```

Returns the full 2D pixel matrix.

Returns

2D vector of pixels (BGR).

Definition at line 324 of file bmp_image.cpp.

```
325 {
326     return data;
327 }
```

3.2.2.5 getWidth()

```
int32_t BmpImage::getWidth ( )
```

Returns the width of the image.

Returns

Image width in pixels.

Definition at line 314 of file bmp_image.cpp.

```
315 {
316     return width;
317 }
```

3.2.2.6 load()

```
void BmpImage::load (
    const std::string & filename )
```

Loads a BMP image from file.

Loads a BMP image from a file into memory.

Parameters

<i>filename</i>	Path to the BMP file.
-----------------	-----------------------

Exceptions

<i>std::runtime_error</i>	if the file is invalid or unreadable.
---------------------------	---------------------------------------

This function reads the BMP and DIB headers, checks validity, loads pixel data, and stores it in the internal 2D vector.

Parameters

<i>filename</i>	Path to the BMP file.
-----------------	-----------------------

Exceptions

<code>std::runtime_error</code>	If the file cannot be opened or is not a valid BMP.
---------------------------------	---

Definition at line 17 of file `bmp_image.cpp`.

```

18 {
19     std::ifstream in(filename, std::ios::binary);
20     if (!in)
21     {
22         throw std::runtime_error("Unable to open BMP file.");
23     }
24
25     if (!in.read(reinterpret_cast<char*>(&header), sizeof(header)))
26     {
27         throw std::runtime_error("Failed to read header");
28     }
29
30     if (!in.read(reinterpret_cast<char*>(&dibHeader), sizeof(dibHeader)))
31     {
32         throw std::runtime_error("Failed to read dibHeader");
33     }
34
35     if (header.fileType != 0x4D42)
36     {
37         throw std::runtime_error("Not a valid BMP file.");
38     }
39
40     width = dibHeader.width;
41     height = dibHeader.height;
42
43     int rowSize = (width * sizeof(Pixel) + 3) & ~3;
44     int paddingSize = rowSize - width * sizeof(Pixel);
45
46     data.resize(height, std::vector<Pixel>(width));
47
48     in.seekg(header.offsetData, std::ios::beg);
49
50     for (int y = height - 1; y >= 0; --y)
51     {
52         for (int x = 0; x < width; ++x)
53         {
54             in.read(reinterpret_cast<char*>(&data[y][x]), sizeof(Pixel));
55         }
56         in.ignore(paddingSize);
57     }
58 }

```

3.2.2.7 rotate90Clockwise()

```
void BmpImage::rotate90Clockwise ( )
```

Rotates the image 90 degrees clockwise.

This operation swaps the image width and height, and rearranges pixel data accordingly.

Definition at line 66 of file `bmp_image.cpp`.

```

67 {
68     std::vector<std::vector<Pixel>> rotatedData(width, std::vector<Pixel>(height));
69
70     for (int y = 0; y < height; ++y)
71     {

```



```

72         for (int x = 0; x < width; ++x)
73         {
74             rotatedData[x][height - 1 - y] = data[y][x];
75         }
76     }
77     data = rotatedData;
78     std::swap(width, height);
79 }

```

Here is the caller graph for this function:

3.2.2.8 rotate90CounterClockwise()

```
void BmpImage::rotate90CounterClockwise ( )
```

Rotates the image 90 degrees counterclockwise.

Rotates the image 90 degrees counter-clockwise.

This operation swaps the image width and height, and rearranges pixel data accordingly.

Definition at line 87 of file bmp_image.cpp.

```

88 {
89     std::vector<std::vector<Pixel>> rotatedData(width, std::vector<Pixel>(height));
90
91     for (int y = 0; y < height; ++y)
92     {
93         for (int x = 0; x < width; ++x)
94         {
95             rotatedData[width - 1 - x][y] = data[y][x];
96         }
97     }
98     data = rotatedData;
99     std::swap(width, height);
100 }

```

Here is the caller graph for this function:

3.2.2.9 save()

```
void BmpImage::save (
    const std::string & filename )
```

Saves the image to a BMP file.

Saves the current image to a BMP file.

Parameters

<i>filename</i>	Path where the image will be saved.
-----------------	-------------------------------------

Exceptions

<i>std::runtime_error</i>	if saving fails.
---------------------------	------------------

The function writes BMP headers and pixel data to a file. Pixels are written bottom-to-top with row padding as per BMP spec.

Parameters

<i>filename</i>	Path to the output BMP file.
-----------------	------------------------------

Exceptions

<i>std::runtime_error</i>	If the output file cannot be created or opened.
---------------------------	---

Definition at line 111 of file bmp_image.cpp.

```

112 {
113     std::ofstream out(filename, std::ios::binary);
114     if (!out)
115     {
116         throw std::runtime_error("Unable to open output file.");
117     }
118
119     int rowSize = (width * sizeof(Pixel) + 3) & ~3;
120     int paddingSize = rowSize - width * sizeof(Pixel);
121
122     header.fileSize = sizeof(BMPHeader) + sizeof(DIBHeader) + rowSize * height;
123     dibHeader.width = width;
124     dibHeader.height = height;
125
126     out.write(reinterpret_cast<const char*>(&header), sizeof(header));
127     out.write(reinterpret_cast<const char*>(&dibHeader), sizeof(dibHeader));
128
129     for (int y = height - 1; y >= 0; --y)
130     {
131         for (int x = 0; x < width; ++x)
132         {
133             out.write(reinterpret_cast<const char*>(&data[y][x]), sizeof(Pixel));
134         }
135         for (int p = 0; p < paddingSize; ++p)
136         {
137             out.put(0);
138         }
139     }
140 }
```

Here is the caller graph for this function:

3.2.3 Member Data Documentation

3.2.3.1 data

`std::vector<std::vector<Pixel> > BmpImage::data` [private]

2D array of pixel data (BGR format).

Definition at line 22 of file bmp_image.h.

3.2.3.2 dibHeader

`DIBHeader BmpImage::dibHeader` [private]

DIB header with image metadata.

Definition at line 20 of file bmp_image.h.

3.2.3.3 header

```
BMPHeader BmpImage::header [private]
```

BMP file header.

Definition at line 19 of file bmp_image.h.

3.2.3.4 height

```
int32_t BmpImage::height [private]
```

Image dimensions (width x height).

Definition at line 21 of file bmp_image.h.

3.2.3.5 width

```
int32_t BmpImage::width [private]
```

Definition at line 21 of file bmp_image.h.

The documentation for this class was generated from the following files:

- [include/bmp_image.h](#)
- [src/bmp_image.cpp](#)

3.3 DIBHeader Struct Reference

DIB header.

```
#include <struct.h>
```

Public Attributes

- uint32_t [size](#)
- int32_t [width](#)
- int32_t [height](#)
- uint16_t [planes](#)
- uint16_t [bitCount](#)
- uint32_t [compression](#)
- uint32_t [imageSize](#)
- int32_t [xPixelsPerMeter](#)
- int32_t [yPixelsPerMeter](#)
- uint32_t [colorsUsed](#)
- uint32_t [colorsImportant](#)

3.3.1 Detailed Description

DIB header.

Describes the dimensions, color format, compression, and resolution of the bitmap image.

Definition at line 30 of file struct.h.

3.3.2 Member Data Documentation

3.3.2.1 bitCount

```
uint16_t DIBHeader::bitCount
```

Bits per pixel.

Definition at line 36 of file struct.h.

3.3.2.2 colorsImportant

```
uint32_t DIBHeader::colorsImportant
```

Number of important colors.

Definition at line 42 of file struct.h.

3.3.2.3 colorsUsed

```
uint32_t DIBHeader::colorsUsed
```

Number of colors used in the palette.

Definition at line 41 of file struct.h.

3.3.2.4 compression

```
uint32_t DIBHeader::compression
```

Compression method.

Definition at line 37 of file struct.h.

3.3.2.5 height

```
int32_t DIBHeader::height
```

Image height in pixels.

Definition at line 34 of file struct.h.

3.3.2.6 imageSize

```
uint32_t DIBHeader::imageSize
```

Size of the raw image data.

Definition at line 38 of file struct.h.

3.3.2.7 planes

```
uint16_t DIBHeader::planes
```

Number of color planes.

Definition at line 35 of file struct.h.

3.3.2.8 size

```
uint32_t DIBHeader::size
```

Size of this header.

Definition at line 32 of file struct.h.

3.3.2.9 width

```
int32_t DIBHeader::width
```

Image width in pixels.

Definition at line 33 of file struct.h.

3.3.2.10 xPixelsPerMeter

```
int32_t DIBHeader::xPixelsPerMeter
```

Horizontal resolution.

Definition at line 39 of file struct.h.

3.3.2.11 yPixelsPerMeter

```
int32_t DIBHeader::yPixelsPerMeter
```

Vertical resolution.

Definition at line 40 of file struct.h.

The documentation for this struct was generated from the following file:

- [include/struct.h](#)

3.4 Pixel Struct Reference

A single pixel in BGR format.

```
#include <struct.h>
```

Public Attributes

- [uint8_t blue](#)
- [uint8_t green](#)
- [uint8_t red](#)

3.4.1 Detailed Description

A single pixel in BGR format.

Represents the color of a pixel using blue, green, and red channels.

Definition at line 51 of file struct.h.

3.4.2 Member Data Documentation

3.4.2.1 blue

```
uint8_t Pixel::blue
```

Blue color component.

Definition at line 53 of file struct.h.

3.4.2.2 green

```
uint8_t Pixel::green
```

Green color component.

Definition at line 54 of file struct.h.

3.4.2.3 red

```
uint8_t Pixel::red
```

Red color component.

Definition at line 55 of file struct.h.

The documentation for this struct was generated from the following file:

- [include/struct.h](#)

Chapter 4

File Documentation

4.1 include/bmp_image.h File Reference

```
#include "struct.h"
#include <vector>
#include <iostream>
```

Include dependency graph for bmp_image.h: This graph shows which files directly or indirectly include this file:

Classes

- class [BmpImage](#)
Class for loading, manipulating, and saving BMP images.

4.2 include/struct.h File Reference

```
#include <stdint>
```

Include dependency graph for struct.h: This graph shows which files directly or indirectly include this file:

Classes

- struct [BMPHeader](#)
BMP file header.
- struct [DIBHeader](#)
DIB header.
- struct [Pixel](#)
A single pixel in BGR format.

4.3 include/threads.h File Reference

```
#include <thread>
#include "bmp_image.h"
```

Include dependency graph for threads.h: This graph shows which files directly or indirectly include this file:

Functions

- void [thread_1](#) ([BmpImage](#))
Thread function that rotates the image.
- void [thread_2](#) ([BmpImage](#))
This function rotates the image and calls the Gaussian filter.

4.3.1 Function Documentation

4.3.1.1 [thread_1\(\)](#)

```
void thread_1 (
    BmpImage image )
```

Thread function that rotates the image.

Parameters

<i>image</i>	A copy of the BmpImage to be processed.
--------------	---

Thread function that rotates the image.

This function is intended to be run in a separate thread. It performs a 90-degree clockwise rotation of the input image and saves the result to "rotated_90.bmp".

Parameters

<i>image</i>	A copy of the BmpImage object to be rotated and saved.
--------------	--

Definition at line 11 of file threads.cpp.

```
12 {
13     image.rotate90Clockwise();
14     std::cout << "Rotated_90\n";
15     image.save("rotated_90.bmp");
16 }
```

Here is the call graph for this function:

4.3.1.2 [thread_2\(\)](#)

```
void thread_2 (
    BmpImage image )
```

This function rotates the image and calls the Gaussian filter.

Parameters

<i>image</i>	A copy of the BmpImage to be processed.
--------------	---

This function rotates the image and calls the Gaussian filter.

This function is intended to be run in a separate thread. It performs a 90-degree counter-clockwise rotation of the image, applies a Gaussian blur filter with a given radius and sigma, and saves two intermediate results: one after rotation and one after filtering.

Parameters

<i>image</i>	A copy of the BmpImage object to be transformed and saved. The function does not modify the original image outside the thread.
--------------	--

Definition at line 28 of file threads.cpp.

```

29 {
30     image.rotate90CounterClockwise();
31     std::cout << "Rotated_un_90\n";
32     image.save("rotated_un90.bmp");
33
34     image.gaussFilter(10, 5);
35     std::cout << "Gaussed\n";
36     image.save("filter.bmp");
37 }
```

Here is the call graph for this function:

4.4 src/bmp_image.cpp File Reference

```

#include <thread>
#include "include/bmp_image.h"
#include <fstream>
#include <cmath>
Include dependency graph for bmp_image.cpp:
```

4.5 src/threads.cpp File Reference

```

#include "include/threads.h"
Include dependency graph for threads.cpp:
```

Functions

- void [thread_1](#) ([BmpImage](#) image)
Rotates the image 90 degrees clockwise and saves the result.
- void [thread_2](#) ([BmpImage](#) image)
Rotates the image 90 degrees counter-clockwise, applies a Gaussian filter, and saves the results.

4.5.1 Function Documentation

4.5.1.1 thread_1()

```
void thread_1 (
    BmpImage image )
```

Rotates the image 90 degrees clockwise and saves the result.

Thread function that rotates the image.

This function is intended to be run in a separate thread. It performs a 90-degree clockwise rotation of the input image and saves the result to "rotated_90.bmp".

Parameters

<i>image</i>	A copy of the BmpImage object to be rotated and saved.
--------------	--

Definition at line 11 of file threads.cpp.

```
12 {  
13     image.rotate90Clockwise();  
14     std::cout << "Rotated_90\n";  
15     image.save("rotated_90.bmp");  
16 }
```

Here is the call graph for this function:

4.5.1.2 thread_2()

```
void thread_2 (  
    BmpImage image )
```

Rotates the image 90 degrees counter-clockwise, applies a Gaussian filter, and saves the results.

This function rotates the image and calls the Gaussian filter.

This function is intended to be run in a separate thread. It performs a 90-degree counter-clockwise rotation of the image, applies a Gaussian blur filter with a given radius and sigma, and saves two intermediate results: one after rotation and one after filtering.

Parameters

<i>image</i>	A copy of the BmpImage object to be transformed and saved. The function does not modify the original image outside the thread.
--------------	--

Definition at line 28 of file threads.cpp.

```
29 {  
30     image.rotate90CounterClockwise();  
31     std::cout << "Rotated_un_90\n";  
32     image.save("rotated_un90.bmp");  
33  
34     image.gaussFilter(10, 5);  
35     std::cout << "Gaussed\n";  
36     image.save("filter.bmp");  
37 }
```

Here is the call graph for this function:

Index

- bitCount
 - DIBHeader, 16
- blue
 - Pixel, 18
- BMPHeader, 5
 - fileSize, 5
 - fileType, 5
 - offsetData, 6
 - reserved1, 6
 - reserved2, 6
- BmpImage, 6
 - createGaussianKernel, 7
 - data, 14
 - dibHeader, 14
 - gaussFilter, 8
 - getHeight, 10
 - getPixel, 10
 - getWidth, 11
 - header, 14
 - height, 15
 - load, 11
 - rotate90Clockwise, 12
 - rotate90CounterClockwise, 13
 - save, 13
 - width, 15
- colorsImportant
 - DIBHeader, 16
- colorsUsed
 - DIBHeader, 16
- compression
 - DIBHeader, 16
- createGaussianKernel
 - BmpImage, 7
- data
 - BmpImage, 14
- DIBHeader, 15
 - bitCount, 16
 - colorsImportant, 16
 - colorsUsed, 16
 - compression, 16
 - height, 16
 - imageSize, 17
 - planes, 17
 - size, 17
 - width, 17
 - xPixelsPerMeter, 17
 - yPixelsPerMeter, 18
- dibHeader
 - BmpImage, 14
- fileSize
 - BMPHeader, 5
- fileType
 - BMPHeader, 5
- gaussFilter
 - BmpImage, 8
- getHeight
 - BmpImage, 10
- getPixel
 - BmpImage, 10
- getWidth
 - BmpImage, 11
- green
 - Pixel, 19
- header
 - BmpImage, 14
- height
 - BmpImage, 15
 - DIBHeader, 16
- imageSize
 - DIBHeader, 17
- include/bmp_image.h, 21
- include/struct.h, 21
- include/threads.h, 21
- load
 - BmpImage, 11
- offsetData
 - BMPHeader, 6
- Pixel, 18
 - blue, 18
 - green, 19
 - red, 19
- planes
 - DIBHeader, 17
- red
 - Pixel, 19
- reserved1
 - BMPHeader, 6
- reserved2
 - BMPHeader, 6
- rotate90Clockwise
 - BmpImage, 12

- rotate90CounterClockwise
 - BmpImage, [13](#)
- save
 - BmpImage, [13](#)
- size
 - DIBHeader, [17](#)
- src/bmp_image.cpp, [23](#)
- src/threads.cpp, [23](#)
- thread_1
 - threads.cpp, [23](#)
 - threads.h, [22](#)
- thread_2
 - threads.cpp, [25](#)
 - threads.h, [22](#)
- threads.cpp
 - thread_1, [23](#)
 - thread_2, [25](#)
- threads.h
 - thread_1, [22](#)
 - thread_2, [22](#)
- width
 - BmpImage, [15](#)
 - DIBHeader, [17](#)
- xPixelsPerMeter
 - DIBHeader, [17](#)
- yPixelsPerMeter
 - DIBHeader, [18](#)